

Tutorial del proyecto

Este documento recorre el notebook celda por celda. Antes de cada bloque de código hay una explicación detallada de qué hace y por qué, y debajo aparece su resultado real. Al final están las conclusiones.

Proyecto: clasificación de intención en español ecuatoriano coloquial aplicada a un sistema multiagente. Manuel Pillapa. 2026.

Sistema Multiagente de Accesibilidad para Adultos Mayores

Clasificación de intención en español ecuatoriano coloquial

Problema

Los adultos mayores tienen dificultad para usar asistentes digitales. Hablan de forma coloquial, con modismos ecuatorianos, y no usan las palabras clave que esperan los sistemas tradicionales. El reto es entender la intención detrás de la frase, no las palabras exactas. Si el sistema confunde una emergencia con una consulta trivial, las consecuencias para el usuario pueden ser graves.

El objetivo de este trabajo es clasificar de forma automática la intención de una frase para enrutarla al agente especialista correcto.

Intenciones

El sistema distingue cinco intenciones:

- `MEDICATION_HEALTH` : medicación, dosis, horarios, síntomas.
- `RECIPE_MULTIMEDIA` : recetas y guías de cocina paso a paso.
- `FAMILY_COMMUNICATION` : enviar avisos o mensajes a familiares.
- `EMERGENCY` : caídas, dolor intenso, peligro en el entorno.
- `SMALL_TALK` : saludos y conversación trivial.

Metodología

El proyecto sigue el flujo estándar de un problema de machine learning supervisado de clasificación de texto:

1. Definición del problema y de las clases objetivo.

2. Carga y exploracion de los datos, con nube de palabras y frecuencias.
3. Division estratificada en entrenamiento y prueba.
4. Extraccion de características con dos representaciones, TF-IDF y embeddings.
5. Entrenamiento de un modelo base y un modelo mejorado con la misma cabeza.
6. Optimizacion de hiperparametros con validacion cruzada y curva de aprendizaje.
7. Evaluacion estadística con accuracy, F1 macro, curva ROC y AUC.
8. Analisis de errores del mejor modelo.
9. Persistencia del modelo entrenado.
10. Aplicacion en un sistema multiagente y comparacion de estrategias de ruteo.

1. Preparacion del entorno

Explicacion del codigo

Esta celda solo sirve para preparar el entorno. El comando `!pip install` corre una instalacion desde dentro del notebook, y la opcion `-q` la deja silenciosa para que no llene la pantalla.

Se instalan dos cosas que Google Colab no trae de fabrica: `sentence-transformers`, que es la libreria que descarga y ejecuta el modelo MiniLM para generar los embeddings, y `wordcloud`, que dibuja la nube de palabras del analisis exploratorio. Si trabajas en local y ya las tienes instaladas, esta celda se puede saltar sin problema.

```
In [54]: # Para correr en google colab
!pip install sentence-transformers wordcloud -q
```

Explicacion del codigo

Aqui se importa todo lo que el proyecto va a usar, agrupado por proposito. De manejo de datos vienen `pandas` y `numpy`. De graficos vienen `matplotlib` y `seaborn`. De scikit-learn se traen cuatro grupos: la division y la validacion cruzada con `train_test_split`, `cross_val_score` y `learning_curve`; los dos representadores de texto, `TfidfVectorizer` y mas abajo el embedder; el modelo `LogisticRegression`; y las metricas de evaluacion como `classification_report`, `confusion_matrix`, `roc_curve` y `roc_auc_score`. `joblib` es para guardar el modelo al final.

La parte importante es la funcion `ruta_dato`. Resuelve un problema real: en tu computadora los archivos estan en una carpeta, pero en Google Colab esa carpeta no existe. La funcion recorre dos rutas locales posibles con un bucle y, si encuentra el archivo, devuelve esa ruta; si no lo encuentra en ninguna, arma la URL del archivo en GitHub y la devuelve. Asi el mismo notebook lee los datos correctamente en local y en Colab sin que tengas que cambiar nada a mano.

```
In [55]: import os
import re
from collections import Counter
```

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, cross_val_score, learning
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.metrics import (
    accuracy_score, f1_score, classification_report, confusion_matrix,
    roc_curve, auc, roc_auc_score
)
from sklearn.preprocessing import label_binarize
import joblib

# Rutas. Funciona en Local y en Google Colab.
# Para Colab, reemplaza USUARIO_GITHUB por tu usuario una vez publicado el repo
USUARIO_GITHUB = 'mpillapa'
REPO_GITHUB = 'accessibility-agents'
URL_BASE = f'https://raw.githubusercontent.com/{USUARIO_GITHUB}/{REPO_GITHUB}/r

def ruta_dato(relativa):
    # Devuelve la ruta local si el archivo existe, o la URL del repositorio si
    for candidata in [os.path.join '..', relativa), relativa]:
        if os.path.exists(candidata):
            return candidata
    return f'{URL_BASE}/{relativa}'

```

2. Carga y exploración del dataset

El dataset propio tiene 415 frases en español ecuatoriano coloquial, balanceado con 83 frases por intención. La definición completa de cada clase con ejemplos está en `data/intents/intents_schema.md` del repositorio.

Explicación del código

`pd.read_csv` carga el dataset en un DataFrame, que es como una tabla de Excel manejable desde Python. Con `len(df)` se cuenta el número de filas, o sea de frases.

Las dos líneas siguientes preparan etiquetas que se reutilizan en todo el notebook.

`INTENCIONES` toma los valores únicos de la columna de intención y los ordena alfabéticamente con `sorted`. Ordenarlos es importante porque garantiza que las matrices de confusión y los gráficos siempre usen el mismo orden de clases, lo que hace los resultados comparables entre celdas. `ETIQUETAS_CORTAS` corta cada nombre antes del guion bajo para tener versiones cortas que no se amontonen en los ejes de los gráficos.

Resultado: confirma que hay 415 frases y muestra con `df.head()` las primeras filas, cada una con su texto y su intención.

```

In [56]: # Carga el dataset desde local o desde GitHub según el entorno
df = pd.read_csv(ruta_dato('data/intents/dataset.csv'))

```

```

print(f'Total de frases: {len(df)}')

# Lista ordenada de las 5 clases; se reutiliza en graficos y matrices
INTENCIONES = sorted(df['intent'].unique())
# Nombres cortos para que no se encimen en los ejes
ETIQUETAS_CORTAS = [e.split('_')[0] for e in INTENCIONES]

df.head()

```

Total de frases: 415

```

Out[56]:

```

	intent	text
0	MEDICATION_HEALTH	Oye mijito ¿ya me toca tomar la pastillita del...
1	MEDICATION_HEALTH	¿A qué hora me corresponde el jarabe que me re...
2	MEDICATION_HEALTH	Tengo un dolorcito en la rodilla desde ayer ¿s...
3	FAMILY_COMMUNICATION	Quiero mandarle un mensajito a mi hijo Carlos ...
4	FAMILY_COMMUNICATION	¿Puedes avisarle a mi hija María que ya tomé l...

Explicacion del codigo

`value_counts()` cuenta cuantas frases hay de cada intencion y `sort_index()` las deja en orden alfabetico para que coincidan con el resto del notebook. Eso se imprime y luego se dibuja como grafico de barras.

El bucle `for barra in ax.patches` recorre cada barra del grafico y le escribe encima el numero exacto de frases con `ax.text`, calculando la posicion horizontal en el centro de la barra y la vertical justo arriba. Es un detalle de presentacion para que el grafico se lea sin tener que estimar la altura.

Resultado: las cinco barras quedan casi en 83 cada una. El dataset esta balanceado, y eso valida usar el accuracy como metrica principal, porque ninguna clase domina y no engana el promedio.

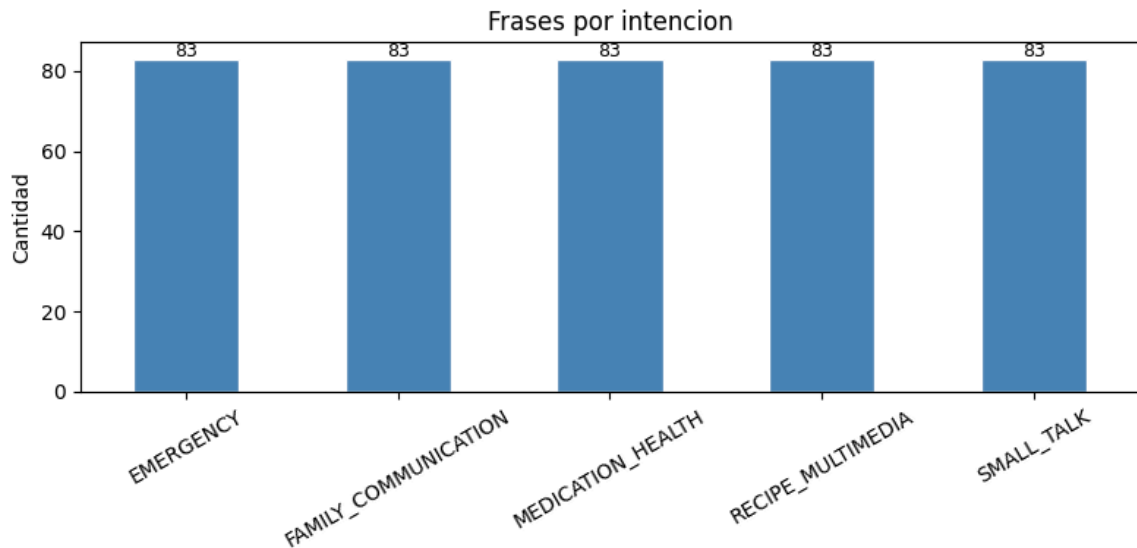
```

In [57]: # Cuenta cuantas frases hay por intencion, en orden alfabetico
conteo = df['intent'].value_counts().sort_index()
print(conteo.to_string())

fig, ax = plt.subplots(figsize=(8, 4))
conteo.plot(kind='bar', ax=ax, color='steelblue', edgecolor='white')
ax.set_title('Frases por intencion')
ax.set_ylabel('Cantidad')
ax.set_xlabel('')
ax.tick_params(axis='x', rotation=30)
# Escribe el numero exacto encima de cada barra
for barra in ax.patches:
    ax.text(barra.get_x() + barra.get_width()/2, barra.get_height() + 0.5,
            str(int(barra.get_height())), ha='center', fontsize=9)
plt.tight_layout()
plt.show()

```

```
intent
EMERGENCY      83
FAMILY_COMMUNICATION  83
MEDICATION_HEALTH  83
RECIPE_MULTIMEDIA  83
SMALL_TALK      83
```



Explicacion del codigo

La primera linea crea una columna nueva con la longitud de cada frase. `str.split()` parte la frase en palabras y `str.len()` cuenta cuantas hay. Luego, con una comprension de lista, se arma una lista de arreglos, uno por intencion, con las longitudes de sus frases.

Esa lista se pasa a `ax.boxplot`, que dibuja un diagrama de caja por intencion. Cada caja muestra la mediana, el rango donde cae la mitad central de los datos y los valores atipicos, asi se ve de un vistazo si una intencion tiende a frases mas largas o mas cortas.

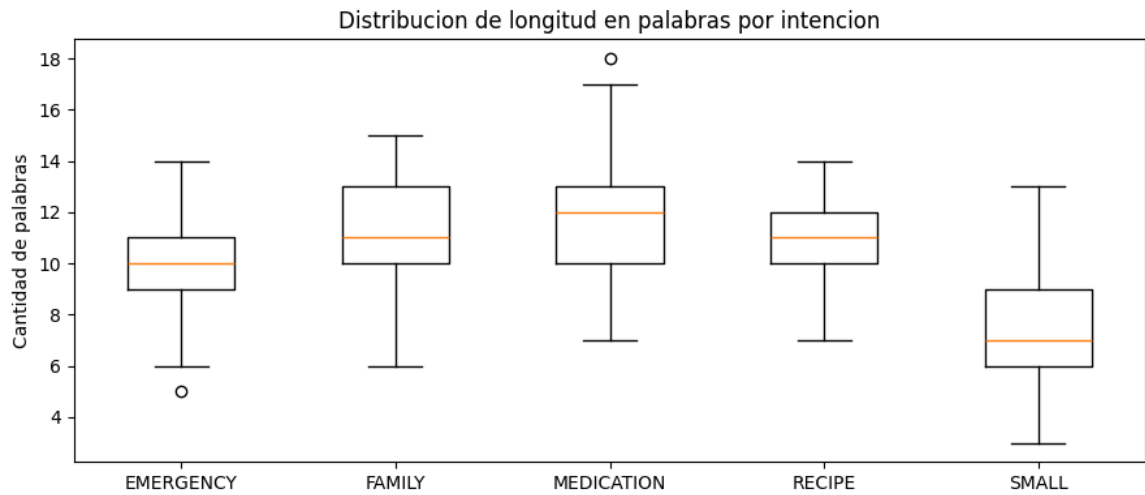
Resultado: las frases son cortas en todas las intenciones, con longitudes parecidas. La leccion es que con tan poco texto por frase, la forma de representar cada frase pesa muchisimo, porque hay poca informacion donde apoyarse.

```
In [58]: # Longitud de cada frase en numero de palabras
df['longitud'] = df['text'].str.split().str.len()
# Una lista de longitudes por cada intencion, para el boxplot
datos_longitud = [df[df['intent'] == i]['longitud'].values for i in INTENCIONES]

fig, ax = plt.subplots(figsize=(9, 4))
ax.boxplot(datos_longitud, labels=ETIQUETAS_CORTAS)
ax.set_title('Distribucion de longitud en palabras por intencion')
ax.set_ylabel('Cantidad de palabras')
plt.tight_layout()
plt.show()
```

C:\Users\Manuel Pillapa\AppData\Local\Temp\ipykernel_27624\2621007068.py:7: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dropped in 3.11.

```
ax.boxplot(datos_longitud, labels=ETIQUETAS_CORTAS)
```



Explicacion del codigo

Este bucle recorre las cinco intenciones y, para cada una, filtra sus frases con `df[df['intent'] == intencion]` y toma tres al azar con `sample(3, random_state=42)`. El `random_state=42` hace que siempre salgan las mismas tres frases, así el ejemplo es reproducible y no cambia cada vez que corres la celda.

Sirve para una inspeccion cualitativa: ver con tus propios ojos como es realmente el lenguaje del dataset antes de modelar.

Resultado: imprime frases coloquiales reales como "oye mijito, ya me toca la pastillita", que muestran el modismo ecuatoriano que el modelo va a tener que entender.

```
In [59]: for intencion in INTENCIONES:
          print()
          print(intencion)
          muestra = df[df['intent'] == intencion]['text'].sample(3, random_state=42)
          for texto in muestra:
              print(f' - {texto}')
```

EMERGENCY

- Necesito que vengan ya me está pasando algo
- Me caí en el baño y no me puedo levantar ayúdame por favor
- Me siento muy mal el corazón late rápido y me falta el aire

FAMILY_COMMUNICATION

- Marca a mi sobrina que hace meses no hablo con ella
- Quiero mandarle un mensajito a mi hijo Carlos que está en Quito
- ¿Hay algún aviso de la familia que deba saber?

MEDICATION_HEALTH

- Siempre me confundo ¿son dos comprimidos blancos o uno blanco y uno amarillo?
- Oye mijito ¿ya me toca tomar la pastillita del corazón?
- Listo ya me tomé el calmante ¿lo registras?

RECIPE_MULTIMEDIA

- Espera espera ¿me repites lo del refrito que no entendí?
- ¿Me lees la receta del locro de papas pasito a pasito?
- Dice una libra de carne ¿eso cuánto es en gramos más o menos?

SMALL_TALK

- ¿Puedes hacer cualquier cosa o tienes límites?
- Buenos días mijito ¿cómo amaneciste hoy?
- Oye hace tanto viento que la ventana no para de sonar

3. Analisis exploratorio: nube de palabras y frecuencias

El dominio es texto coloquial. Se visualizan las palabras que dominan el corpus y si hay solapamiento de vocabulario entre intenciones. Mucho solapamiento anticipa dificultades para un clasificador basado en conteo de palabras como TF-IDF.

Explicacion del codigo

Esta celda hace el analisis exploratorio del texto. Primero define `STOPWORDS_ES`, un conjunto de palabras vacias como "de", "la" o "que", que aparecen mucho pero no aportan significado. La funcion `tokenizar` limpia cada texto: con una expresion regular quita los signos de puntuacion, pasa todo a minusculas, y se queda solo con las palabras que no son vacias y que tienen al menos cuatro letras, para descartar ruido.

Con esas palabras limpias, `Counter` cuenta frecuencias. La nube de palabras se dibuja con `generate_from_frequencies`, donde el tamano de cada palabra es proporcional a cuantas veces aparece. Al lado se grafican las 10 mas frecuentes en barras horizontales. Al final, un bucle recorre cada intencion y lista sus 5 palabras mas comunes por separado.

Resultado: se observa que varias palabras se repiten entre intenciones. Ese solapamiento de vocabulario es justamente la pista de que un modelo que solo cuenta palabras, como TF-IDF, va a confundir clases, y es el argumento para probar embeddings despues.

```
In [60]: from wordcloud import WordCloud
```

```
# Palabras vacías que no aportan significado y se descartan del análisis
STOPWORDS_ES = {
    'de', 'que', 'el', 'la', 'en', 'es', 'me', 'mi', 'te', 'le', 'se',
    'un', 'una', 'los', 'las', 'al', 'del', 'ya', 'no', 'si', 'por',
    'con', 'para', 'como', 'hay', 'lo', 'su', 'tu', 'yo', 'pero', 'mas',
    'muy', 'he', 'ha', 'son', 'tan', 'fue', 'ser', 'han', 'esta', 'este'
}
```

```
def tokenizar(texto):
    # Quita puntuación, pasa a minúsculas y deja solo palabras de 4 letras o más
    texto = re.sub(r'^\w\s', '', texto.lower())
    return [p for p in texto.split() if p not in STOPWORDS_ES and len(p) >= 4]
```

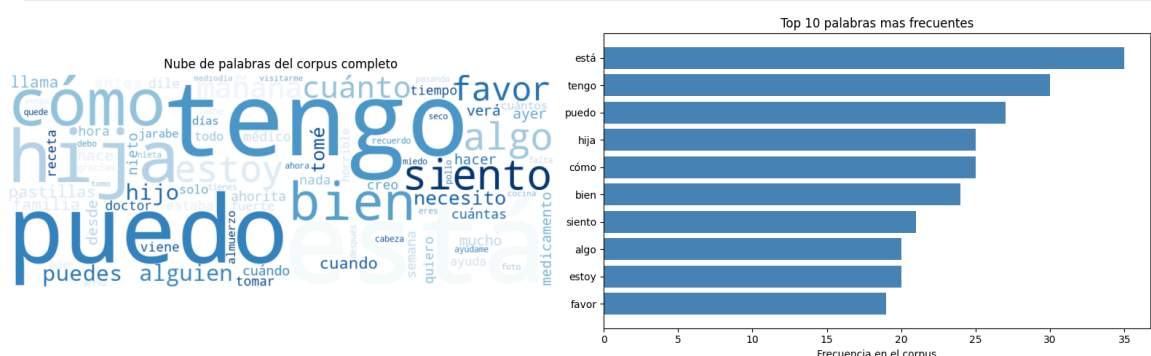
```
# Frecuencia de palabras en todo el corpus
todo_tokens = tokenizar(' '.join(df['text'].tolist()))
frecuencias_global = Counter(todo_tokens)
```

```
# Nube de palabras: el tamaño de cada palabra es proporcional a su frecuencia
wc = WordCloud(width=900, height=350, background_color='white', colormap='Blues')
wc.generate_from_frequencies(frecuencias_global)
```

```
fig, axes = plt.subplots(1, 2, figsize=(16, 5))
axes[0].imshow(wc, interpolation='bilinear')
axes[0].axis('off')
axes[0].set_title('Nube de palabras del corpus completo')
```

```
# Las 10 palabras más frecuentes, en barras horizontales
top10 = frecuencias_global.most_common(10)
palabras_top, freqs_top = zip(*top10)
axes[1].barh(range(len(palabras_top)), list(reversed(freqs_top)), color='steelblue')
axes[1].set_yticks(range(len(palabras_top)))
axes[1].set_yticklabels(list(reversed(palabras_top)), fontsize=10)
axes[1].set_xlabel('Frecuencia en el corpus')
axes[1].set_title('Top 10 palabras más frecuentes')
plt.tight_layout()
plt.show()
```

```
# Las 5 palabras más frecuentes de cada intención por separado
print('Palabras más frecuentes por intención:')
for intencion in INTENCIONES:
    tokens = tokenizar(' '.join(df[df['intent'] == intencion]['text'].tolist()))
    top5 = [p for p, _ in Counter(tokens).most_common(5)]
    print(f'{intencion:<22}: {"", ".join(top5)}')
```



Palabras mas frecuentes por intencion:

EMERGENCY	: puedo, siento, estoy, está, alguien
FAMILY_COMMUNICATION	: hija, hijo, familia, dile, puedes
MEDICATION_HEALTH	: tomé, pastillas, tengo, medicamento, mañana
RECIPE_MULTIMEDIA	: cómo, cuánto, receta, bien, tiempo
SMALL_TALK	: está, gracias, todo, eres, nada

4. Division en entrenamiento y prueba

Se divide 80 por ciento para entrenamiento y 20 por ciento para prueba, de forma estratificada para que cada intencion conserve su proporcion en ambos conjuntos. La semilla fija en 42 hace el resultado reproducible.

Explicacion del codigo

`train_test_split` separa los datos en dos grupos. `test_size=0.2` reserva el 20 por ciento para prueba y deja el 80 por ciento para entrenar.

`stratify=df['intent']` es la opcion clave: obliga a que cada intencion conserve su proporcion en ambos grupos, asi el conjunto de prueba no queda, por ejemplo, sin emergencias. `random_state=42` fija la division para que sea siempre la misma y el experimento se pueda repetir.

Las ultimas lineas arman un DataFrame que pone lado a lado cuantas frases de cada intencion quedaron en entrenamiento y en prueba, como verificacion de que el balance se mantuvo.

Resultado: 332 frases de entrenamiento y 83 de prueba, con las cinco clases repartidas de forma proporcional en los dos grupos.

```
In [61]: # 80% entrenamiento, 20% prueba. stratify mantiene la proporcion de cada
# intencion en ambos grupos; random_state fija la division para reproducirla
X_entrenamiento, X_prueba, y_entrenamiento, y_prueba = train_test_split(
    df['text'].tolist(),
    df['intent'].tolist(),
    test_size=0.2,
    stratify=df['intent'],
    random_state=42
)

print(f'Frases de entrenamiento: {len(X_entrenamiento)}')
print(f'Frases de prueba: {len(X_prueba)}')

# Verifica que el balance entre clases se mantuvo en ambos grupos
resumen_split = pd.DataFrame({
    'Train': pd.Series(y_entrenamiento).value_counts().sort_index(),
    'Test': pd.Series(y_prueba).value_counts().sort_index()
})
resumen_split
```

Frases de entrenamiento: 332

Frases de prueba: 83

Out[61]:

	Train	Test
EMERGENCY	66	17
FAMILY_COMMUNICATION	67	16
MEDICATION_HEALTH	66	17
RECIPE_MULTIMEDIA	66	17
SMALL_TALK	67	16

5. Extraccion de características y modelo base: TF-IDF + Logistic Regression

Primera representacion: TF-IDF con unigramas y bigramas. Convierte cada frase en un vector de frecuencias de palabras y pares de palabras, ponderado por que tan informativa es cada termino en el corpus.

No se eliminan stopwords. En frases cortas y coloquiales, palabras como me, ya u oye ayudan a distinguir intenciones, asi que quitarlas perderia señal.

La cabeza del clasificador es Logistic Regression con class_weight balanceado, para que ninguna intencion domine por cantidad.

Explicacion del codigo

Aqui se entrena el primer modelo. `TfidfVectorizer` convierte texto en numeros y sus parametros importan: `gram_range=(1, 2)` le dice que use palabras sueltas y tambien pares de palabras consecutivas, lo que captura expresiones como "me cai"; `min_df=2` ignora terminos que aparecen en una sola frase para reducir ruido; `strip_accents='unicode'` y `lowercase=True` normalizan el texto quitando tildes y mayusculas.

`fit_transform` se aplica solo al entrenamiento: aprende el vocabulario y transforma. Sobre la prueba se usa solo `transform`, sin volver a aprender, para no contaminar el test con informacion que no deberia conocer. Encima va `LogisticRegression`, con `max_iter=1000` para darle iteraciones suficientes de convergencia, `class_weight='balanced'` para que ninguna clase pese mas que otra, y `random_state=42` para reproducibilidad.

`classification_report` imprime precision, recall y F1 por cada intencion. Precision es de lo que predijo como clase X cuanto era realmente X; recall es de todo lo que era X cuanto detecto; F1 combina ambas.

Resultado: accuracy cercano al 85 por ciento. Buen punto de partida, con debilidad en small talk y emergencia.

In [62]:

```
# gram_range (1,2): palabras sueltas y pares. min_df=2: ignora terminos raros.  
# No se quitan stopwords: en frases cortas palabras como "ya" u "oye" dan señal
```

```

vectorizador_tfidf = TfidfVectorizer(ngram_range=(1, 2), min_df=2, lowercase=True)

# fit_transform aprende el vocabulario solo del entrenamiento;
# en el test solo se transforma, para no filtrarle informacion
X_tfidf_train = vectorizador_tfidf.fit_transform(X_entrenamiento)
X_tfidf_test = vectorizador_tfidf.transform(X_prueba)

# class_weight balanced evita que una clase pese mas que otra al entrenar
logreg_tfidf = LogisticRegression(max_iter=1000, class_weight='balanced', random_state=42)
logreg_tfidf.fit(X_tfidf_train, y_entrenamiento)

predicciones_tfidf = logreg_tfidf.predict(X_tfidf_test)
print(classification_report(y_prueba, predicciones_tfidf, target_names=INTENCIONES))

```

	precision	recall	f1-score	support
EMERGENCY	0.83	0.88	0.86	17
FAMILY_COMMUNICATION	0.94	1.00	0.97	16
MEDICATION_HEALTH	0.88	0.82	0.85	17
RECIPE_MULTIMEDIA	0.74	0.82	0.78	17
SMALL_TALK	0.92	0.75	0.83	16
accuracy			0.86	83
macro avg	0.86	0.86	0.86	83
weighted avg	0.86	0.86	0.86	83

Explicacion del codigo

`confusion_matrix` cuenta, para cada combinacion de intencion real y predicha, cuantas frases cayeron ahi. El parametro `labels=INTENCIONES` fija el orden de filas y columnas para que coincida con el resto del notebook.

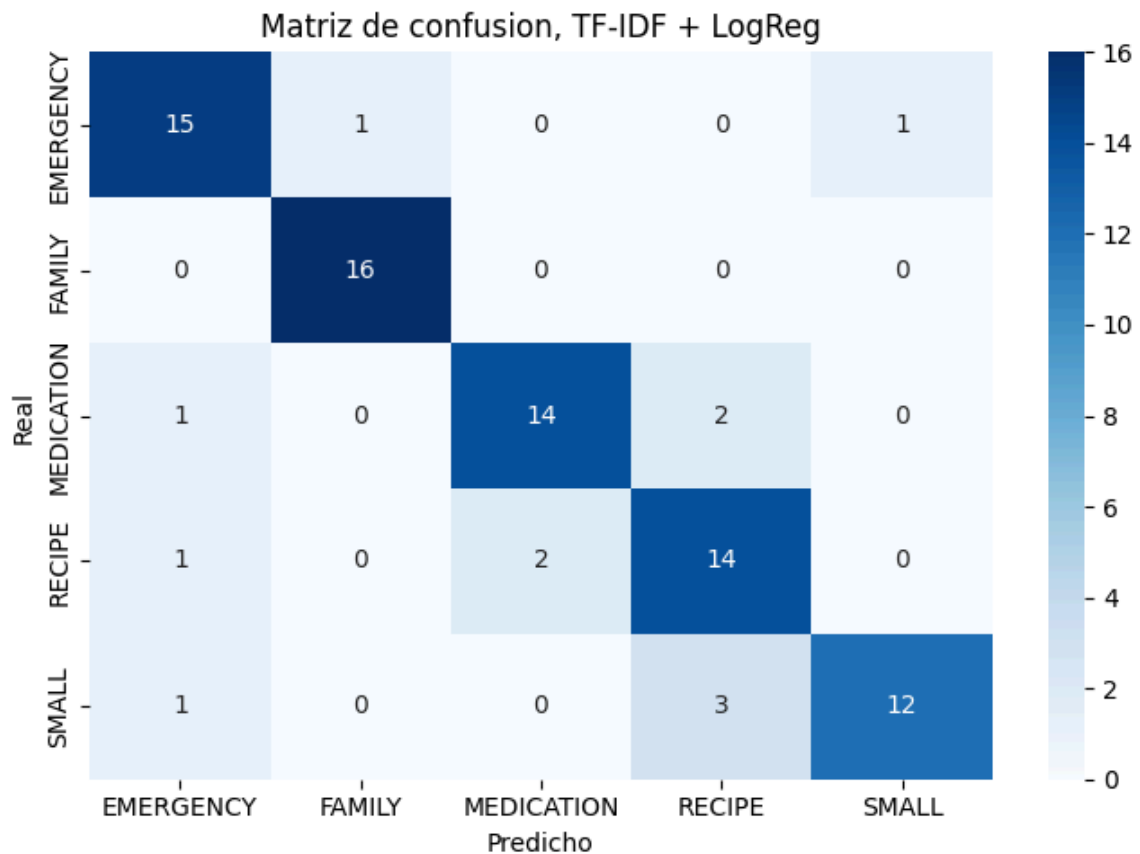
`sns.heatmap` la dibuja como mapa de calor. `annot=True` escribe el numero en cada celda y `fmt='d'` lo muestra como entero. Las filas son la intencion real y las columnas la predicha, asi que la diagonal son los aciertos y todo lo de fuera son errores.

Resultado: se ven las confusiones concentradas fuera de la diagonal, sobre todo small talk que se va a recetas y casos de emergencia mal ubicados.

```

In [63]: # Filas: intencion real. Columnas: predicha. La diagonal son los aciertos
cm_tfidf = confusion_matrix(y_prueba, predicciones_tfidf, labels=INTENCIONES)
plt.figure(figsize=(7, 5))
sns.heatmap(cm_tfidf, annot=True, fmt='d', cmap='Blues', xticklabels=ETIQUETAS)
plt.title('Matriz de confusion, TF-IDF + LogReg')
plt.ylabel('Real')
plt.xlabel('Predicho')
plt.tight_layout()
plt.show()

```



Explicacion del codigo

Esta celda evalua el modelo TF-IDF de forma mas honesta con validacion cruzada de 5 particiones. `cross_val_score` divide el entrenamiento en 5 partes, entrena con 4 y mide con la quinta, rotando, y promedia.

El detalle critico es `make_pipeline`. Mete el vectorizador y el modelo juntos para que en cada particion el TF-IDF se reajuste solo con las 4 partes de entrenamiento de esa vuelta. Si se vectorizara todo de golpe antes de dividir, el vocabulario se aprenderia tambien de la parte de validacion, lo que seria fuga de datos y daria un resultado enganosamente alto.

Resultado: el accuracy cruzado sale mas bajo que el del test unico, lo cual es esperable y mas realista sobre como generaliza el modelo con datos nuevos.

```
In [64]: # make_pipeline reajusta el TF-IDF en cada particion
# Sin pipeline el vocabulario se aprenderia del dataset completo y habria fuga
puntajes_cv_tfidf = cross_val_score(
    make_pipeline(
        TfidfVectorizer(ngram_range=(1, 2), min_df=2, lowercase=True, strip_accents='unicode'),
        LogisticRegression(max_iter=1000, class_weight='balanced', random_state=42)
    ),
    X_entrenamiento, y_entrenamiento, cv=5, scoring='accuracy'
)
print(f'CV 5-fold TF-IDF, accuracy: {puntajes_cv_tfidf.mean():.4f} mas menos {puntajes_cv_tfidf.std():.4f}')
```

CV 5-fold TF-IDF, accuracy: 0.7440 mas menos 0.0192

Analisis del modelo base

El modelo TF-IDF con Logistic Regression alcanza un accuracy cercano al 85 por ciento. Es un punto de partida razonable, pero falla en tres patrones:

- Small talk contra recetas. Preguntas como cuanto falta para navidad no tienen vocabulario de cocina, pero al no tener vocabulario propio el modelo las asigna a la clase con mayor solapamiento superficial.
- Medicacion contra comunicacion familiar. Frases sin nombres de medicamentos ni sintomas explicitos se apoyan en co ocurrencias de palabras poco discriminativas.
- Emergencia con urgencia implicita. Frases como que alguien venga ahorita no tienen terminos de emergencia explicitos, asi que TF-IDF no capta la urgencia.

La causa de fondo es que TF-IDF mide presencia y frecuencia de palabras, no significado. Dos frases opuestas pueden parecer similares si comparten preposiciones, y dos frases equivalentes pueden parecer distintas si usan sinonimos coloquiales.

6. Modelo mejorado: embeddings MiniLM + Logistic Regression

Segunda representacion: embeddings del modelo paraphrase-multilingual-MiniLM-L12-v2. En lugar de contar palabras, codifica cada frase en un vector de 384 dimensiones que captura su significado. Frases parecidas en sentido quedan cerca en el espacio vectorial aunque usen palabras distintas.

La cabeza es la misma Logistic Regression. Al cambiar solo el representador, la diferencia de resultados mide el aporte de la representacion.

Explicacion del codigo

`SentenceTransformer` descarga y carga en memoria el modelo MiniLM preentrenado, que pesa alrededor de 120 MB. Este modelo es el que sabe convertir una frase en un embedding, es decir en un vector que representa su significado.

Se carga en una variable global una sola vez, porque cargarlo es lo costoso; una vez en memoria, generar embeddings es rapido. Este modelo no se entrena con nuestros datos, viene ya entrenado con millones de frases en varios idiomas.

```
In [65]: from sentence_transformers import SentenceTransformer

# Se carga el modelo multilingue MiniLM para generar embeddings de las frases en el
modelo_minilm = SentenceTransformer('sentence-transformers/paraphrase-multiling

Loading weights:   0%|          | 0/199 [00:00<?, ?it/s]
```

Explicacion del codigo

Esta es la celda del segundo modelo. `modelo_minilm.encode` convierte cada frase en un vector de 384 numeros que captura su significado. Se hace por separado para entrenamiento y prueba. `X_emb_train.shape[1]` confirma esa dimension de 384.

Un punto importante de metodología esta en el comentario: el encoder es fijo y preentrenado, no aprende nada de nuestros datos, así que calcular los embeddings de una vez no genera fuga de datos, a diferencia de lo que pasaba con TF-IDF.

Después se entrena exactamente la misma `LogisticRegression` de antes, con los mismos parámetros, sobre estos embeddings, y se imprime el reporte. Mantener la misma cabeza es lo que permite atribuir cualquier mejora únicamente al cambio de representación.

Resultado: accuracy cercano al 92 por ciento, claramente por encima del TF-IDF.

```
In [66]: # El encoder es preentrenado y fijo, no se entrena en este notebook
# Solo se usa para generar embeddings de las frases
X_emb_train = modelo_minilm.encode(X_entrenamiento, show_progress_bar=True)
X_emb_test  = modelo_minilm.encode(X_prueba, show_progress_bar=True)

print(f'Dimension de cada embedding: {X_emb_train.shape[1]}')

logreg_emb = LogisticRegression(max_iter=1000, class_weight='balanced', random_
logreg_emb.fit(X_emb_train, y_entrenamiento)

predicciones_emb = logreg_emb.predict(X_emb_test)
print(classification_report(y_prueba, predicciones_emb, target_names=INTENCIONES))
```

```
Batches:  0%|          | 0/11 [00:00<?, ?it/s]
Batches:  0%|          | 0/3 [00:00<?, ?it/s]
Dimension de cada embedding: 384
```

	precision	recall	f1-score	support
EMERGENCY	1.00	0.88	0.94	17
FAMILY_COMMUNICATION	0.88	0.94	0.91	16
MEDICATION_HEALTH	1.00	0.88	0.94	17
RECIPE_MULTIMEDIA	0.85	1.00	0.92	17
SMALL_TALK	0.88	0.88	0.88	16
accuracy			0.92	83
macro avg	0.92	0.92	0.92	83
weighted avg	0.92	0.92	0.92	83

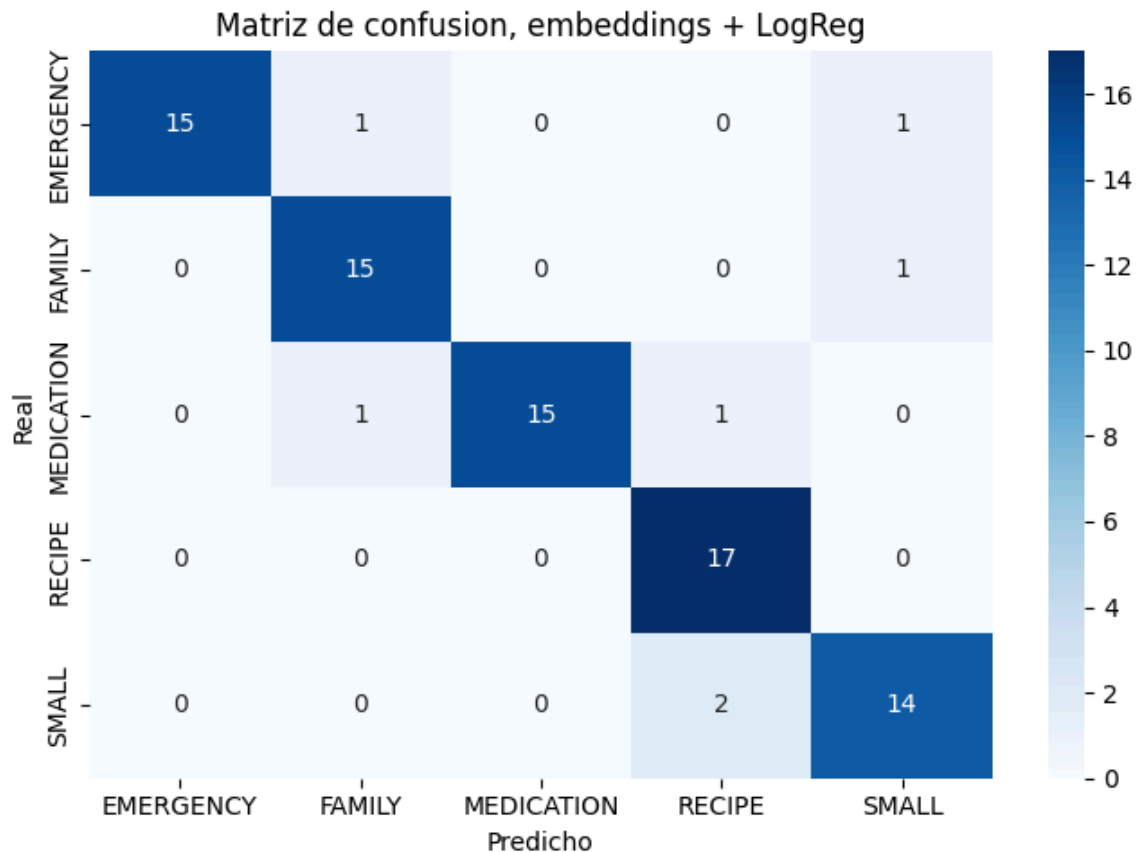
Explicación del código

Misma matriz de confusión de antes, ahora para el modelo de embeddings. Filas reales, columnas predichas, diagonal de aciertos.

Resultado: la diagonal se ve más fuerte que en TF-IDF y hay menos números fuera de ella, sobre todo mejora en emergencia y medicación, que son las clases más delicadas.

```
In [67]: # Crea la matriz de confusion para embeddings + Logistic Regression
cm_emb = confusion_matrix(y_prueba, predicciones_emb, labels=INTENCIONES)
plt.figure(figsize=(7, 5))
sns.heatmap(cm_emb, annot=True, fmt='d', cmap='Blues', xticklabels=ETIQUETAS_CO
plt.title('Matriz de confusion, embeddings + LogReg')
plt.ylabel('Real')
plt.xlabel('Predicho')
```

```
plt.tight_layout()
plt.show()
```



Explicacion del codigo

Validacion cruzada de 5 particiones para el modelo de embeddings, igual que se hizo con TF-IDF. Aqui no hace falta pipeline, porque el encoder es fijo y precalcular los embeddings no introduce fuga; por eso se pasa directamente la matriz de embeddings al `cross_val_score`.

Resultado: el accuracy cruzado tambien sube respecto al de TF-IDF, lo que confirma que la ventaja de los embeddings es consistente y no producto del azar de una sola division.

```
In [68]: # cross_val_score con embeddings no necesita pipeline porque el encoder es fijo
puntajes_cv_emb = cross_val_score(
    LogisticRegression(max_iter=1000, class_weight='balanced', random_state=42),
    X_emb_train, y_entrenamiento, cv=5, scoring='accuracy'
)
print(f'CV 5-fold embeddings, accuracy: {puntajes_cv_emb.mean():.4f} mas menos
```

CV 5-fold embeddings, accuracy: 0.8735 mas menos 0.0399

Analisis del modelo con embeddings

El modelo con embeddings MiniLM mejora en todas las intenciones. El accuracy sube a cerca del 92 por ciento y el F1 macro tambien. La mayor ganancia esta en emergencia y medicacion.

Mejora porque MiniLM fue preentrenado con pares de frases semanticamente relacionadas en varios idiomas. Entiende que necesito ayuda ya y marca el 911 significan lo mismo aunque no compartan palabras. TF-IDF las ve como vectores sin relacion.

Donde persisten errores es en small talk. Preguntas meta como que tanto sabes activan representaciones de otros dominios. Y una taquicardia enviada a comunicacion familiar sugiere que el modelo prioriza la intencion de llamar a alguien sobre la urgencia clinica.

7. Optimizacion de hiperparametros y curva de aprendizaje

El hiperparametro a optimizar es C de Logistic Regression, el inverso de la fuerza de regularizacion. Un C pequeño regulariza mas, sube el sesgo y baja la varianza. Un C grande hace lo contrario.

Se barre C en escala logaritmica de 0.001 a 1000 y se mide el error por validacion cruzada de 5 particiones. La curva de aprendizaje muestra como cambia el error de entrenamiento y de validacion al aumentar el tamaño del dataset. Se usa log loss porque con embeddings el modelo clasifica su propio set de forma perfecta, asi que el log loss revela la confianza de la prediccion donde el F1 ya no informa.

Explicacion del codigo

Esta celda hace dos analisis y los dibuja lado a lado.

El primero es la optimizacion del hiperparametro C. `np.logspace(-3, 3, 20)` genera 20 valores de C en escala logaritmica, de 0.001 a 1000. El bucle entrena un modelo con cada C, guarda el error de entrenamiento y mide el error de validacion cruzada con F1 macro. Luego `np.argmin` busca el C que da menos error de validacion, que es el que mejor generaliza. C controla la regularizacion: chico simplifica el modelo, grande lo deja ajustarse mas.

El segundo es la curva de aprendizaje, con `learning_curve`. Entrena el modelo con tamanos de dataset crecientes, del 10 al 100 por ciento, y mide el error en cada caso. Usa `neg_log_loss` como metrica: scikit-learn la devuelve en negativo, por eso se le invierte el signo. Se eligio log loss y no exactitud porque, con 384 dimensiones, el modelo acierta su entrenamiento de forma perfecta y la exactitud no mostraria nada; el log loss mide la confianza de la prediccion y si revela la dinamica. El `fill_between` dibuja la banda de variacion alrededor de la curva de validacion.

Resultado: a la izquierda se ve el C optimo marcado; a la derecha, como el error de validacion baja y se acerca al de entrenamiento conforme hay mas datos, senal de que no hay sobreajuste grave.

```
In [69]: # Optimizacion del hiperparametro C
C_values = np.logspace(-3, 3, 20)
train_scores_C, cv_scores_C = [], []
```



```

# Entrenamos un modelo para cada valor de C y guarda el F1 macro de entrenamiento
for C in C_values:
    modelo_C = LogisticRegression(C=C, max_iter=1000, class_weight='balanced',
    modelo_C.fit(X_emb_train, y_entrenamiento)
    train_scores_C.append(modelo_C.score(X_emb_train, y_entrenamiento))
    cv = cross_val_score(modelo_C, X_emb_train, y_entrenamiento, cv=5, scoring=
    cv_scores_C.append(cv.mean())

# Convertimos F1 macro a loss (1 - F1) para graficar
train_loss_C = [1 - s for s in train_scores_C]
cv_loss_C     = [1 - s for s in cv_scores_C]
idx_optimo    = int(np.argmin(cv_loss_C))
C_optimo      = C_values[idx_optimo]

fig, axes = plt.subplots(1, 2, figsize=(15, 5))

axes[0].semilogx(C_values, cv_loss_C, 'b-o', markersize=5, label='CV Loss')
axes[0].semilogx(C_values, train_loss_C, 'r-o', markersize=5, label='Training Loss')
axes[0].axvline(C_optimo, color='green', linestyle='--', label=f'C optimo = {C_optimo}')
axes[0].set_xlabel('C, inverso de la regularizacion')
axes[0].set_ylabel('Loss, 1 menos F1 macro')
axes[0].set_title('Optimizacion del hiperparametro C')
axes[0].legend()

# Curva de aprendizaje con Log Loss.
# Con embeddings de 384 dimensiones y pocas muestras el modelo clasifica su
# propio set de forma perfecta, asi que el F1 de entrenamiento es 1.0 siempre.
# El log loss mide la confianza de la prediccion, no solo si acierta, por lo
# que da una curva informativa aunque el accuracy sea 100 por ciento.
train_sizes, tr_sc, cv_sc = learning_curve(
    LogisticRegression(C=1.0, max_iter=1000, class_weight='balanced', random_state=0),
    X_emb_train, y_entrenamiento,
    train_sizes=np.linspace(0.1, 1.0, 10),
    cv=5, scoring='neg_log_loss'
)

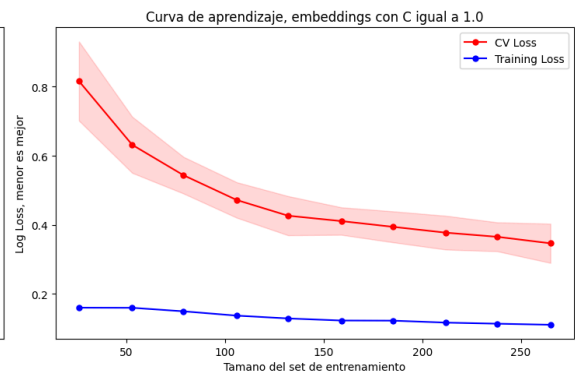
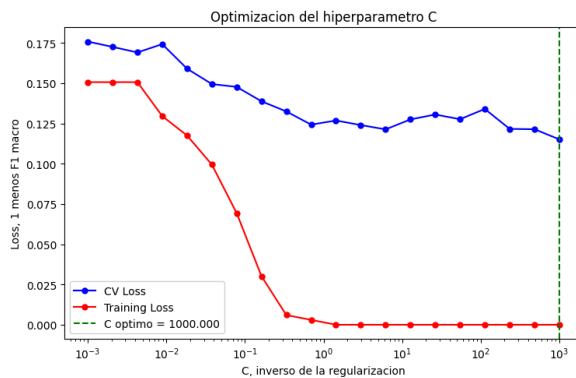
tr_loss     = -tr_sc.mean(axis=1)
cv_loss_lc  = -cv_sc.mean(axis=1)
cv_std_lc   = cv_sc.std(axis=1)

axes[1].plot(train_sizes, cv_loss_lc, 'r-o', markersize=5, label='CV Loss')
axes[1].fill_between(train_sizes, cv_loss_lc - cv_std_lc, cv_loss_lc + cv_std_lc, color='red')
axes[1].plot(train_sizes, tr_loss, 'b-o', markersize=5, label='Training Loss')
axes[1].set_xlabel('Tamaño del set de entrenamiento')
axes[1].set_ylabel('Log Loss, menor es mejor')
axes[1].set_title('Curva de aprendizaje, embeddings con C igual a 1.0')
axes[1].legend()

plt.tight_layout()
plt.show()

print(f'C optimo: {C_optimo:.4f}')
print(f'F1 macro CV en C optimo: {max(cv_scores_C):.4f}')

```



C optimo: 1000.0000

F1 macro CV en C optimo: 0.8849

8. Comparacion estadistica: curva ROC y AUC

Ademas de accuracy y F1, se evalua con curva ROC y AUC. Como hay cinco clases, se usa la estrategia One vs Rest: para cada intencion se traza una curva ROC tratando esa clase como positiva y el resto como negativa. El AUC macro es el promedio de los cinco valores. El AUC resume la capacidad de separar clases en todos los umbrales de decision, no solo en el de por defecto.

Explicacion del codigo

Calcula las metricas finales de los dos modelos. `accuracy_score` da el porcentaje total de aciertos. `f1_score` con `average='macro'` promedia el F1 de las cinco clases dandoles el mismo peso, lo que es justo en un dataset balanceado.

Con esos numeros, mas los promedios de validacion cruzada que ya se calcularon, arma una tabla comparativa en un DataFrame para ver los dos modelos lado a lado.

Resultado: la fila de embeddings supera a la de TF-IDF en accuracy, F1 macro y validacion cruzada.

```
In [80]: # Comparacion de resultados entre TF-IDF y embeddings
acc_tfidf = accuracy_score(y_prueba, predicciones_tfidf)
acc_emb   = accuracy_score(y_prueba, predicciones_emb)
f1_tfidf  = f1_score(y_prueba, predicciones_tfidf, average='macro')
f1_emb    = f1_score(y_prueba, predicciones_emb, average='macro')

tabla_comparacion = pd.DataFrame({
    'Modelo':    ['TF-IDF + LogReg', 'Embeddings + LogReg'],
    'Accuracy':  [round(acc_tfidf, 4), round(acc_emb, 4)],
    'F1 macro':  [round(f1_tfidf, 4), round(f1_emb, 4)],
    'CV media':  [round(puntajes_cv_tfidf.mean(), 4), round(puntajes_cv_emb.mean(), 4)],
    'CV std':    [round(puntajes_cv_tfidf.std(), 4), round(puntajes_cv_emb.std(), 4)]
})
tabla_comparacion
```

Out[80]:

	Modelo	Accuracy	F1 macro	CV media	CV std
0	TF-IDF + LogReg	0.8554	0.8561	0.7440	0.0192
1	Embeddings + LogReg	0.9157	0.9156	0.8735	0.0399

Explicacion del codigo

Esta celda hace la comparacion estadistica con curvas ROC y AUC. Como hay cinco clases, se usa One vs Rest: `label_binarize` convierte las etiquetas en columnas de ceros y unos, una por clase. `predict_proba` da la probabilidad que el modelo asigna a cada clase.

El bucle dibuja, para cada modelo y cada intencion, su curva ROC con `roc_curve` y calcula su area con `auc`. La curva ROC enfrenta la tasa de verdaderos positivos contra la de falsos positivos; mientras mas pegada a la esquina superior izquierda, mejor. Al final, `roc_auc_score` con `average='macro'` y `multi_class='ovr'` calcula el AUC macro, el promedio de las cinco areas, y arma una tabla.

El AUC resume la capacidad de separar clases considerando todos los umbrales de decision, no solo el de por defecto, por eso es una medida mas robusta que el accuracy.

Resultado: el modelo de embeddings tiene mayor AUC macro, confirmando que su ventaja no depende de un umbral particular.

```
In [71]: # One vs Rest: binariza las etiquetas, una columna de 0 y 1 por clase
y_bin = label_binarize(y_prueba, classes=INTENCIONES)
# Probabilidad que cada modelo asigna a cada clase
y_prob_tfidf = logreg_tfidf.predict_proba(X_tfidf_test)
y_prob_emb = logreg_emb.predict_proba(X_emb_test)

fig, axes = plt.subplots(1, 2, figsize=(14, 6))
colores_roc = plt.cm.tab10.colors

# Una curva ROC por intencion, en cada uno de los dos modelos
for ax, y_prob, titulo in [(axes[0], y_prob_tfidf, 'TF-IDF + LogReg'), (axes[1], y_prob_emb, 'Embeddings + LogReg')]:
    for i, intencion in enumerate(INTENCIONES):
        fpr, tpr, _ = roc_curve(y_bin[:, i], y_prob[:, i])
        auc_score = auc(fpr, tpr)
        ax.plot(fpr, tpr, color=colores_roc[i], label=f'{intencion.split("_")[0]} vs Rest')
        ax.plot([0, 1], [0, 1], 'k--', alpha=0.4, label='Aleatorio')
        ax.set_xlabel('Tasa de falsos positivos')
        ax.set_ylabel('Tasa de verdaderos positivos')
        ax.set_title(f'Curva ROC One vs Rest, {titulo}')
        ax.legend(fontsize=8, loc='lower right')

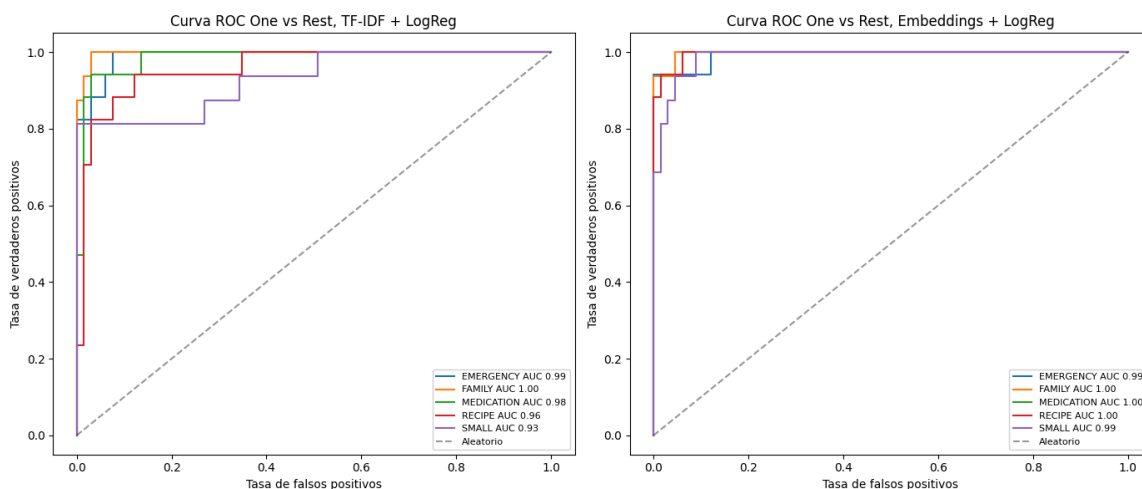
plt.tight_layout()
plt.show()

# AUC macro: promedio del area bajo la curva de las 5 clases
auc_macro_tfidf = roc_auc_score(y_bin, y_prob_tfidf, average='macro', multi_class='ovr')
auc_macro_emb = roc_auc_score(y_bin, y_prob_emb, average='macro', multi_class='ovr')
```

```

tabla_auc = pd.DataFrame({
    'Modelo': ['TF-IDF + LogReg', 'Embeddings + LogReg'],
    'Accuracy': [round(acc_tfidf, 4), round(acc_emb, 4)],
    'F1 macro': [round(f1_tfidf, 4), round(f1_emb, 4)],
    'AUC macro': [round(auc_macro_tfidf, 4), round(auc_macro_emb, 4)],
})
tabla_auc

```



Out[71]:

	Modelo	Accuracy	F1 macro	AUC macro
0	TF-IDF + LogReg	0.8554	0.8561	0.9717
1	Embeddings + LogReg	0.9157	0.9156	0.9938

Interpretacion de la comparacion estadistica

El modelo con embeddings gana en accuracy, F1 macro y AUC macro. El AUC confirma que separa mejor las clases en todos los umbrales de decision, no solo en el punto por defecto.

El costo extra de los embeddings vale la pena. En un sistema de accesibilidad una confusion entre emergencia y small talk puede tener consecuencias graves para el adulto mayor, y la ganancia reduce justo esos errores de alta consecuencia. Ademas, calcular el embedding tarda menos de medio segundo en CPU, un tiempo despreciable frente a los segundos que tarda el agente en responder.

9. Analisis de errores del mejor modelo

Se listan todas las frases mal clasificadas por el mejor modelo, junto con la confianza con que se equivoco. Sirve para distinguir errores claros de ambigüedades reales del lenguaje.

Explicacion del codigo

Esta celda analiza los errores del mejor modelo. Primero un `if` elige cual fue el mejor comparando el F1 de los dos, y se queda con sus predicciones y sus probabilidades.

Luego un bucle recorre a la vez la frase, su intencion real, la predicha y las probabilidades, usando `zip`. Cuando la real no coincide con la predicha, guarda esa frase como error junto con la confianza, que es la probabilidad maxima que el modelo dio, o sea que tan seguro estaba al equivocarse. Todo se junta en un DataFrame.

Resultado: 7 errores de 83. Ver la confianza de cada error permite distinguir un fallo claro, donde el modelo estaba muy seguro, de una duda legitima entre dos clases parecidas.

```
In [72]: # Elige el mejor modelo por F1 macro y se queda con sus predicciones
if f1_emb >= f1_tfidf:
    predicciones_mejor = predicciones_emb
    probas_mejor       = logreg_emb.predict_proba(X_emb_test)
    nombre_mejor       = 'Embeddings + LogReg'
else:
    predicciones_mejor = predicciones_tfidf
    probas_mejor       = logreg_tfidf.predict_proba(X_tfidf_test)
    nombre_mejor       = 'TF-IDF + LogReg'

print(f'Mejor modelo: {nombre_mejor}')

# Recorre el test y guarda solo las frases mal clasificadas
errores = []
for frase, real, pred, probas in zip(X_prueba, y_prueba, predicciones_mejor, probas_mejor):
    if real != pred:
        # confianza = probabilidad maxima, que tan seguro estaba el modelo al equivocarse
        errores.append({'frase': frase, 'real': real, 'predicha': pred, 'confianza': probas[pred].max()})

df_errores = pd.DataFrame(errores)
print(f'Errores: {len(df_errores)} de {len(X_prueba)}')
df_errores
```

Mejor modelo: Embeddings + LogReg
Errores: 7 de 83

Out[72]:

	frase	real	predicha	confianza
0	Por favor que alguien venga ahorita mismo	EMERGENCY	SMALL_TALK	0.713
1	¿Cuánto falta para navidad?	SMALL_TALK	RECIPE_MULTIMEDIA	0.536
2	Anota que el frasco del diurético está casi va...	MEDICATION_HEALTH	RECIPE_MULTIMEDIA	0.684
3	El prospecto dice una cosa y el doctor dijo ot...	MEDICATION_HEALTH	FAMILY_COMMUNICATION	0.455
4	¿Hay algo especial en la familia para las fies...	FAMILY_COMMUNICATION	SMALL_TALK	0.801
5	Me está dando taquicardia y no puedo controlarla	EMERGENCY	FAMILY_COMMUNICATION	0.497
6	¿Cómo sabes tanto si eres una máquina?	SMALL_TALK	RECIPE_MULTIMEDIA	0.517

Explicacion del codigo

Toma la tabla de errores y la agrupa con `groupby` por la combinacion de intencion real y predicha, contando cuantas veces ocurrio cada par con `size`. Luego ordena de mayor a menor.

Sirve para ver patrones: no es lo mismo que los errores esten repartidos al azar a que haya un par de clases que el modelo confunde sistematicamente.

Resultado: las confusiones que mas se repiten son small talk hacia recetas, y casos de emergencia que terminan en otras clases.

```
In [73]: if len(df_errores) > 0:
    pares = df_errores.groupby(['real', 'predicha']).size().reset_index(name='cantidad')
    print('Pares de confusion, real hacia predicha:')
    print(pares.to_string(index=False))
```

```
Pares de confusion, real hacia predicha:
      real      predicha  cantidad
0  SMALL_TALK  RECIPE_MULTIMEDIA      2
1  EMERGENCY  FAMILY_COMMUNICATION      1
2  EMERGENCY      SMALL_TALK      1
3  FAMILY_COMMUNICATION  SMALL_TALK      1
4  MEDICATION_HEALTH  FAMILY_COMMUNICATION      1
5  MEDICATION_HEALTH      RECIPE_MULTIMEDIA      1
```

Lectura de los errores

El mejor modelo comete alrededor de 7 errores en 83 frases, cerca del 8 por ciento. La mayoría son ambigüedades legítimas:

- Que alguien venga ahorita mismo, sin contexto, podría ser cualquier intención.
- Una taquicardia suena a llamar a alguien antes que a emergencia inmediata.
- Un frasco casi vacío usa la palabra frasco, que el modelo asocia a recetas.

El único fallo con consecuencia de seguridad es la taquicardia clasificada como comunicación familiar. Ese caso señala la próxima mejora del dataset: agregar más frases de síntomas cardíacos.

10. Guardado del modelo

Se guarda el mejor modelo en disco para que el sistema de agentes lo cargue sin reentrenar. No se guarda el encoder de embeddings porque pesa mucho y se recarga aparte desde el código del clasificador.

Explicación del código

Guarda el modelo entrenado para que el sistema de agentes lo use después sin tener que reentrenar. Un `if` arma un diccionario `paquete` distinto según el tipo de modelo ganador: si es de embeddings guarda el clasificador y las etiquetas; si fuera TF-IDF guardaría además el vectorizador.

`joblib.dump` escribe ese paquete a disco en `modelo.joblib`. Un detalle a notar: no se guarda el encoder de embeddings, porque pesa 120 MB; en su lugar, el código del clasificador lo vuelve a cargar aparte cuando hace falta. Así el archivo guardado queda liviano.

Resultado: confirma el tipo de modelo guardado y la lista de etiquetas que reconoce.

```
In [74]: if nombre_mejor == 'Embeddings + LogReg':
          paquete = {'tipo': 'embeddings', 'clasificador': logreg_emb, 'etiquetas': ...
        else:
          paquete = {'tipo': 'tfidf', 'vectorizador': vectorizador_tfidf, 'clasifica...

          joblib.dump(paquete, 'modelo.joblib')
          print(f'Modelo guardado. Tipo: {paquete["tipo"]}')
          print(f'Etiquetas: {paquete["etiquetas"]}')
```

Modelo guardado. Tipo: embeddings

Etiquetas: [np.str_('EMERGENCY'), np.str_('FAMILY_COMMUNICATION'), np.str_('MEDICATION_HEALTH'), np.str_('RECIPE_MULTIMEDIA'), np.str_('SMALL_TALK')]

11. Aplicación: sistema multiagente

El clasificador no es el fin en si mismo. Es el router de un sistema multiagente construido con CrewAI y un modelo de lenguaje local, qwen2.5:7b servido por Ollama. Cada intencion tiene su propio agente especialista.

El sistema corre en local porque necesita Ollama instalado en la maquina. El codigo completo esta en src/agentes.py del repositorio. A continuacion se resume la arquitectura y las tres estrategias de ruteo que probe.

Arquitectura

```
Usuario -> Orchestrator -> [ Medicacion | Recetas | Familia |
Emergencia | Small talk ]
```

Tres estrategias de ruteo

La pregunta de fondo es quien decide a que especialista enviar cada consulta. Probe tres respuestas distintas.

Estrategia B. El modelo de lenguaje del Orchestrator lee la consulta y decide por su cuenta a quien delegar. No usa el clasificador.

```
# Nota de autoria: La estructura base de Los agentes con CrewAI se desarrollo
# con ayuda de Claude. Prompt de referencia que se le dio:
# "Como armo en CrewAI un Orchestrator que delegue la consulta a
agentes
# especialistas segun la intencion, y como expongo mi clasificador de
# scikit-learn como una herramienta del agente?"
def procesar_consulta_v_b(consulta):
    # B: el modelo de lenguaje del Orchestrator razona y delega solo
    crew = crear_crew() # Orchestrator mas 4 especialistas
    return crew.kickoff(inputs={'consulta': consulta})
```

Estrategia A1. El clasificador entrenado decide la intencion y un despachador de Python llama directo al especialista. El modelo de lenguaje no interviene en la decision de ruteo.

```
def procesar_consulta_v_a1(consulta):
    # A1: el clasificador decide y Python despacha directo al
    especialista
    intencion = predecir(consulta)['intencion']
    if intencion == 'MEDICATION_HEALTH':
        return ejecutar(crear_agente_medicacion(), consulta)
    elif intencion == 'RECIPE_MULTIMEDIA':
        return ejecutar(crear_agente_recetas(), consulta)
    elif intencion == 'FAMILY_COMMUNICATION':
        return ejecutar(crear_agente_familia(), consulta)
    elif intencion == 'EMERGENCY':
        return ejecutar(crear_agente_emergencia(), consulta)
    else:
        return 'Hola, en que puedo ayudarte hoy'
```


Estrategia A2. El clasificador se expone como una herramienta del Orchestrator. El modelo de lenguaje decide cuando invocarlo y usa su resultado para delegar. Es un hibrido entre A1 y B.

```
@tool('Clasificador de Intencion')
def herramienta_clasificador(consulta):
    # El clasificador disponible como herramienta del Orchestrator
    r = predecir(consulta)
    return f"Intencion: {r['intencion']} con confianza
    {r['confianza']:.2f}"

def procesar_consulta_v_a2(consulta):
    # A2: el Orchestrator tiene el clasificador como herramienta y
    # decide cuando usarlo
    orchestrator = crear_orchestrator_con_herramienta() # tools=
    [herramienta_clasificador]
    crew = Crew(agents=[orchestrator, medicacion, recetas, familia,
    emergencia], tasks=[tarea])
    return crew.kickoff()
```

En una frase: A1 deja la decision al clasificador, B la deja al modelo de lenguaje, A2 le da el clasificador al modelo de lenguaje como apoyo. La seccion 12 mide cual funciona mejor.

Resultados de la demostracion

Cinco consultas representativas, una por intencion, con la estrategia B:

Consulta	Intencion esperada	Agente que respondio	Correcto
pastillita del corazon	MEDICATION_HEALTH	Especialista en Medicacion	Si
seco de pollo	RECIPE_MULTIMEDIA	Especialista en Recetas	Si
avisale a mi hija	FAMILY_COMMUNICATION	Puente de Comunicacion Familiar	Si
ayudame que me cai	EMERGENCY	Especialista en Medicacion	No
hola mijito	SMALL_TALK	Orchestrator	Parcial

El ruteo por razonamiento del modelo acerto en las frases explicitas, pero fallo en un caso critico: interpreto una caida como consulta de salud en lugar de emergencia. Ese fallo es lo que motiva medir con datos cual estrategia de ruteo es mas confiable, en la siguiente seccion.

12. Comparativa de estrategias de ruteo

Para decidir cual estrategia conviene, evalúe las tres sobre las 83 frases de prueba y guarde los resultados en resultados_evaluacion.csv, un total de 249 filas. Aquí cargo esos resultados y mido latencia y acierto de ruteo.

Estrategia	Como decide el ruteo	Expone la intencion
A1	El clasificador decide y Python despacha	Si
B	El modelo de lenguaje razona y delega	No
A2	El modelo de lenguaje usa el clasificador como herramienta	No

Explicacion del codigo

Empieza la parte de comparacion de las tres estrategias de ruteo. `pd.read_csv` carga `resultados_evaluacion.csv`, que contiene las 83 frases de prueba ya procesadas por las tres estrategias A1, B y A2, con su latencia y su resultado. Eso da 249 filas en total.

Esta seccion no vuelve a ejecutar el sistema de agentes, solo lee resultados ya calculados, por eso si corre en cualquier entorno incluido Colab.

Resultado: confirma que hay 83 frases por cada una de las tres estrategias.

```
In [81]: # Carga el CSV de resultados de evaluacion y muestra un resumen
df_eval = pd.read_csv(ruta_dato('resultados_evaluacion.csv'))
print(f'Total de filas: {len(df_eval)}')
print(df_eval['version'].value_counts().to_string())
df_eval.head()
```

Total de filas: 249

version

A1 83

B 83

A2 83

Out[81]:

	version	indice	consulta	intencion_real	intencion_predicha	
0	A1	0	Quiero mandarle un mensajito a mi hijo Carlos ...	FAMILY_COMMUNICATION	FAMILY_COMMUNICATION	id t 7
1	A1	1	Que venga alguien por favor me siento muy mal	EMERGENCY	EMERGENCY	u
2	A1	2	Creo que ya casi no tengo ibuprofeno ¿cuánto m...	MEDICATION_HEALTH	MEDICATION_HEALTH	En to el i
3	A1	3	¿Cien mililitros de aceite cuántas cucharadas ...	RECIPE_MULTIMEDIA	RECIPE_MULTIMEDIA	Ho co r
4	A1	4	No me acuerdo si ya me tomé la pastilla de la ...	MEDICATION_HEALTH	MEDICATION_HEALTH	E pre

Explicacion del codigo

Primero filtra posibles filas con error, quedandose solo con las de latencia valida. Luego define `resumen_version`, una funcion que para un grupo de datos calcula el numero de frases y la media, mediana, minimo y maximo de latencia.

Con una comprension de diccionario aplica esa funcion a cada estrategia y junta todo en una tabla. La media y la mediana juntas son informativas: si difieren mucho es senal de que hay valores extremos que jalen el promedio.

Resultado: A1 es la mas rapida, A2 la mas lenta, y la diferencia entre media y mediana revela que algunas consultas tardan mucho mas que la mayoría.

```
In [ ]: # Filtra las filas con latencia negativa, que son errores de medicion
df_ok = df_eval[df_eval['latencia_segundos'] >= 0].copy()

# Resumen de latencias por version
def resumen_version(df_v):
    lat = df_v['latencia_segundos']
    return {
```

```

        'n': len(df_v),
        'latencia_media': round(lat.mean(), 2),
        'latencia_mediana': round(lat.median(), 2),
        'latencia_min': round(lat.min(), 2),
        'latencia_max': round(lat.max(), 2),
    }

resumen = {v: resumen_version(df_ok[df_ok['version'] == v]) for v in ['A1', 'B']}
df_resumen = pd.DataFrame(resumen).T
df_resumen

```

```
Out[ ]:
```

	n	latencia_media	latencia_mediana	latencia_min	latencia_max
A1	83.0	3.39	2.47	0.01	21.03
B	83.0	4.48	2.33	0.93	30.44
A2	83.0	11.89	8.91	2.64	34.26

Explicacion del codigo

Calcula el accuracy de ruteo de A1, que es la unica estrategia que expone la intencion predicha de forma estructurada y por tanto se puede medir de forma exacta. La expresion compara la columna predicha con la real y `.mean()` sobre el resultado booleano da directamente la proporcion de aciertos.

Luego desglosa ese acierto por intencion: agrupa por la intencion real y, con `agg`, cuenta los aciertos y el total de cada clase para sacar su accuracy individual.

Resultado: 91.6 por ciento de ruteo correcto. Recetas acierta el 100 por ciento y small talk es la intencion mas dificil.

```

In [77]: df_a1 = df_ok[df_ok['version'] == 'A1'].copy()
# Accuracy de ruteo: proporcion de veces que la predicha coincide con la real
accuracy_a1 = (df_a1['intencion_predicha'] == df_a1['intencion_real']).mean()
print(f'Accuracy de ruteo A1: {accuracy_a1:.1%}')

# Desglose del acierto por cada intencion
df_a1['correcto'] = df_a1['intencion_predicha'] == df_a1['intencion_real']
aciertos = df_a1.groupby('intencion_real')['correcto'].agg(['sum', 'count'])
aciertos['accuracy'] = (aciertos['sum'] / aciertos['count']).round(3)
aciertos.columns = ['correctas', 'total', 'accuracy']
aciertos

```

Accuracy de ruteo A1: 91.6%

Out[77]:

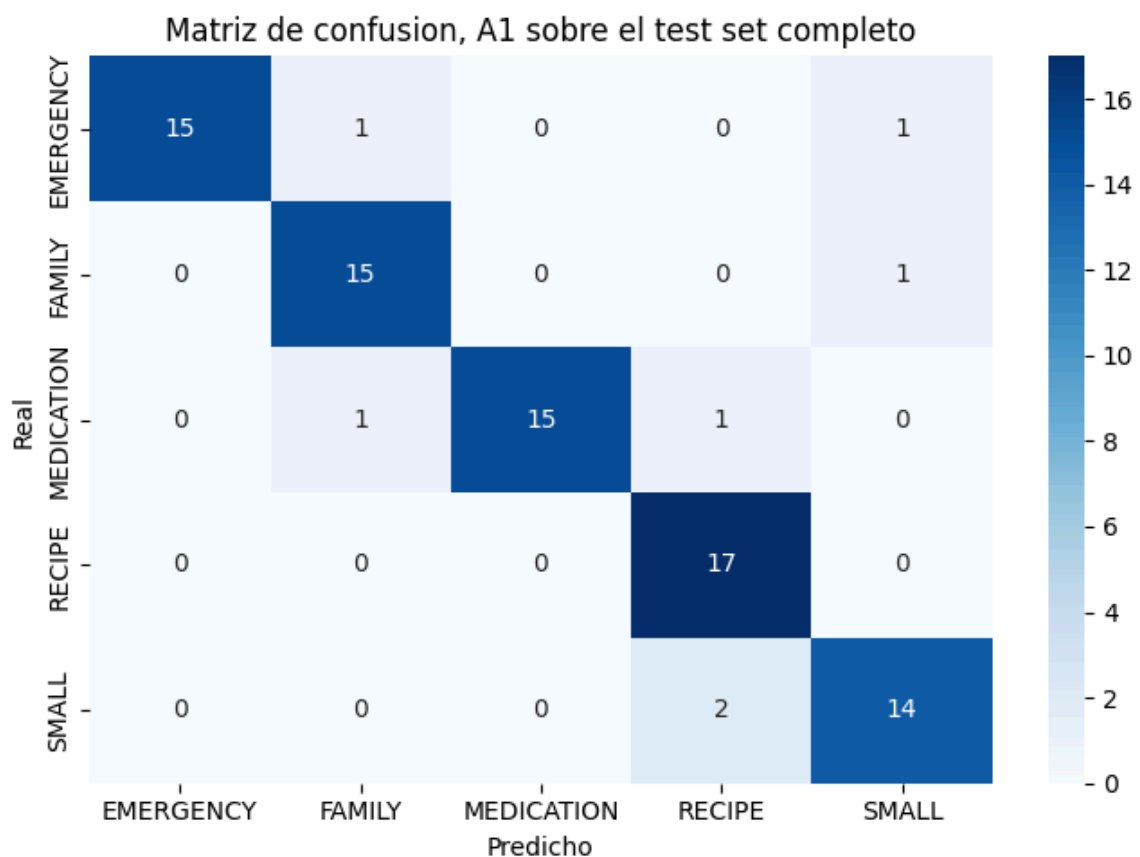
	correctas	total	accuracy
intencion_real			
EMERGENCY	15	17	0.882
FAMILY_COMMUNICATION	15	16	0.938
MEDICATION_HEALTH	15	17	0.882
RECIPE_MULTIMEDIA	17	17	1.000
SMALL_TALK	14	16	0.875

Explicacion del codigo

Matriz de confusion del ruteo de A1 sobre las 83 frases de prueba, dibujada igual que las de los modelos de clasificacion. Confirma visualmente donde se equivoca el ruteo.

Resultado: la gran mayoría de los casos cae en la diagonal, con pocos errores dispersos, coherente con el 91.6 por ciento medido.

```
In [78]: cm = confusion_matrix(df_a1['intencion_real'], df_a1['intencion_predicha'], labels=ETIQUETAS_CORTAS)
plt.figure(figsize=(7, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=ETIQUETAS_CORTAS)
plt.title('Matriz de confusion, A1 sobre el test set completo')
plt.ylabel('Real')
plt.xlabel('Predicho')
plt.tight_layout()
plt.show()
```



Explicacion del codigo

Dibuja dos graficos de latencia uno al lado del otro. El primero es un grafico de barras que pone la media y la mediana de cada estrategia juntas, desplazando las barras media ancho para que no se encimen. El segundo es un boxplot que muestra la distribucion completa de latencias por estrategia, donde se ve el rango y los valores extremos.

Resultado: a simple vista A1 es la mas rapida y compacta, B esta en medio, y A2 es la mas lenta y la mas dispersa, lo que confirma con datos la recomendacion de usar A1.

```
In [79]: versiones = ['A1', 'B', 'A2']
medias = [df_resumen.loc[v, 'latencia_media'] for v in versiones]
medianas = [df_resumen.loc[v, 'latencia_mediana'] for v in versiones]

fig, axes = plt.subplots(1, 2, figsize=(14, 4))

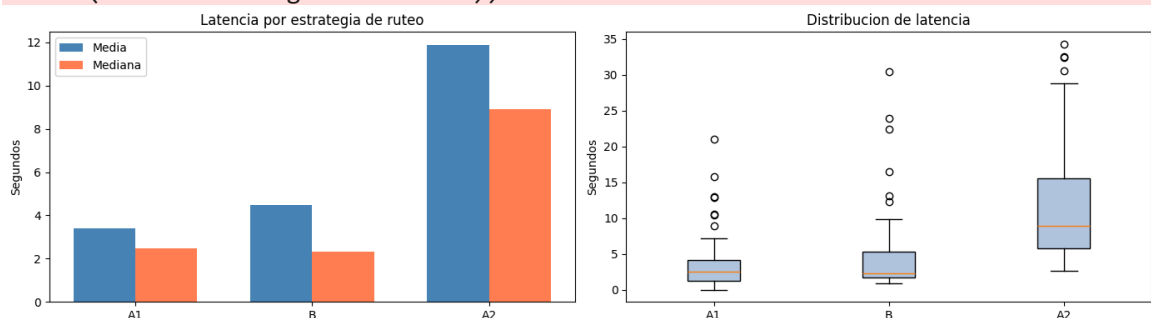
x = np.arange(len(versiones))
ancho = 0.35
axes[0].bar(x - ancho/2, medias, ancho, label='Media', color='steelblue')
axes[0].bar(x + ancho/2, medianas, ancho, label='Mediana', color='coral')
axes[0].set_xticks(x)
axes[0].set_xticklabels(versiones)
axes[0].set_ylabel('Segundos')
axes[0].set_title('Latencia por estrategia de ruteo')
axes[0].legend()

datos_latencia = [df_ok[df_ok['version'] == v]['latencia_segundos'].values for v in versiones]
axes[1].boxplot(datos_latencia, labels=versiones, patch_artist=True, boxprops=dict(facecolor='lightsteelblue'))
axes[1].set_ylabel('Segundos')
axes[1].set_title('Distribucion de latencia')

plt.tight_layout()
plt.show()
```

C:\Users\Manuel Pillapa\AppData\Local\Temp\ipykernel_27624\631303008.py:18: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dropped in 3.11.

```
axes[1].boxplot(datos_latencia, labels=versiones, patch_artist=True, boxprops=dict(facecolor='lightsteelblue'))
```



Conclusiones de la comparativa

Latencia. A1 responde en 3.39 segundos de media. B en 4.48 y A2 en 11.89. A1 es 1.3 veces mas rapida que B y 3.5 veces mas rapida que A2. La ventaja crece en small talk,

donde A1 responde casi al instante porque no llama al modelo de lenguaje.

Accuracy de ruteo. Solo A1 expone la intencion de forma estructurada, asi que solo en A1 se puede medir el acierto de forma exacta: 91.6 por ciento, 76 de 83. B y A2 se evaluan de forma cualitativa sobre una muestra. B acierta alrededor del 72 por ciento en el conjunto completo, con la mayoria de fallos en small talk mal enviado al puente familiar. A2 mejora a B porque el clasificador guia la decision, pero usa la herramienta de forma inconsistente y a veces ignora su salida.

Robustez. A1 es determinista. Ante el mismo texto produce siempre el mismo ruteo, lo que permite auditar y monitorear. B y A2 varian entre ejecuciones y a veces inventan nombres de agentes o devuelven respuestas meta descriptivas poco utiles para el usuario.

Recomendacion. A1 para produccion. Tiene accuracy medible, es determinista, no gasta tokens en el ruteo y sus fallos son conocidos y mejorables agregando datos al dataset. Su limite son las frases ambiguas, que se mitigan con un umbral de confianza: si la confianza es menor a 0.5, el sistema pide al usuario que reformule.

13. Conclusiones generales

1. La representacion importa mas que la cabeza. Con el mismo Logistic Regression, pasar de TF-IDF a embeddings MiniLM subio el F1 macro de 0.86 a 0.92 y el accuracy de 85.5 a 91.6 por ciento. La ganancia se concentra en emergencia y medicacion, las clases de mayor consecuencia.
2. Los embeddings capturan semantica que TF-IDF no ve. Frases de urgencia sin palabras clave explicitas, como pedir que venga alguien ya, se clasifican mejor porque el modelo entiende el significado y no solo la coincidencia de palabras.
3. El costo de los embeddings es despreciable en este uso. La clasificacion tarda menos de medio segundo en CPU, frente a los varios segundos que tarda el modelo de lenguaje del agente. Para un router, los embeddings son la eleccion correcta.
4. Un clasificador dedicado supera al ruteo por razonamiento del modelo de lenguaje. La estrategia A1 es mas rapida, medible y determinista que dejar que el modelo de lenguaje decida. Es la recomendacion para un sistema en produccion.
5. Los errores restantes son ambigüedades legitimas del lenguaje. De 7 errores, 6 son frases que una persona tambien dudaria como clasificar. El unico fallo de seguridad, una taquicardia enviada a comunicacion familiar, indica la siguiente mejora del dataset.

14. Referencias

- Reimers, N. y Gurevych, I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP 2019.
- Modelo de embeddings paraphrase-multilingual-MiniLM-L12-v2. Sentence-Transformers, Hugging Face.
- Pedregosa et al. Scikit-learn: Machine Learning in Python. JMLR 2011.
- CrewAI, framework para sistemas multiagente. <https://docs.crewai.com>
- Ollama, ejecucion local de modelos de lenguaje. <https://ollama.com>
- Modelo de lenguaje local qwen2.5:7b.
- Dataset propio de 415 frases en español ecuatoriano coloquial, 83 por intencion. Ver data/intents/intents_schema.md en el repositorio.